

LGTM 프로젝트 결과보고서

[프로젝트 기본 정보]

- 프로젝트명: LGTM Observability Stack
 - 수행 기간: 2026. 06. 08. ~ 2026. 06. 18. (약 2주 수행)
 - 배포 환경: Cloud VM (IXcloud, Ubuntu 22.04 LTS, Docker Compose 기반)
 - 외부 접속 엔드포인트: <http://1.201.177.179:3000/> (Grafana)
 - 산출물(Git): https://github.com/kinx12399/LGTM_Observability_stack
-

1. 프로젝트 개요

1.1 프로젝트 목적

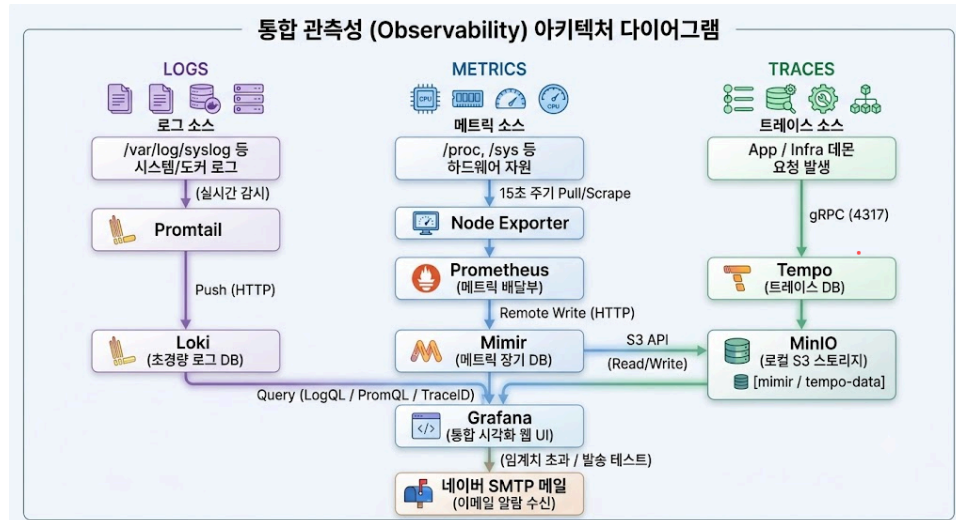
- **통합 관찰성(Observability) 환경 구축:** 단일 Cloud VM 환경 내에서 LGTM(Loki, Grafana, Tempo, Mimir) 스택을 연동하여, 시스템 및 인프라의 로그, 메트릭, 트레이스를 한곳에서 모니터링할 수 있는 고도화된 통합 관찰성 플랫폼을 구현합니다.
- **오픈소스 기반 모니터링 역량 강화:** Promtail, Node Exporter 등의 에이전트를 활용해 데이터를 수집하고, 효율적인 오픈소스 인프라 운영 및 모니터링 아키텍처 설계 능력을 확보하는 것을 목적으로 합니다.

1.2 프로젝트 범위

- **인프라 및 시스템 환경:** Cloud VM (IXcloud, Ubuntu 22.04 LTS) 상에 Docker Compose 기반으로 가상화 환경을 격리 및 구축
 - **LGTM 스택 및 에이전트 구축:**
 - **Grafana:** 통합 모니터링 시각화 및 대시보드 구축, 알람(Alert) 규칙 설정
 - **Loki & Promtail:** 시스템 및 로그 실시간 수집 및 중앙 저장소 구축
 - **Mimir, Prometheus & Node Exporter:** 15초 주기의 하드웨어/자원 메트릭 수집 및 시계열 데이터 저장
 - **Tempo:** 분산 트레이싱 기반 마련 및 gRPC(포트 4317) 연동
 - **안정성 및 검증:** 각 컴포넌트의 Health Check, 스트레스 테스트(부하 테스트)를 통한 디스크 및 시스템 알람 기능 검증, RESOLVED 정책 수립
-

2. 아키텍처 설명

2.1 아키텍처 다이어그램



2.2 컴포넌트 간 데이터 흐름(Logs/Metrics/Traces)

2.2.1 로그 파이프라인: `/var/log/*log` → Promtail → Loki

- **로그 소스:** VM의 `/var/log/*log` (syslog/auth.log/kern.log 등)
- **수집 방식:** Promtail이 파일 tailing 방식으로 실시간 감시 수행
- **전송 방식:** Promtail → Loki로 **HTTP Push** (`/loki/api/v1/push`)
- **가치:** 서비스 장애 시 메트릭과 함께 로그 이벤트를 빠르게 특정 가능

2.2.2 메트릭 파이프라인: Node Exporter → Prometheus → Mimir

- **메트릭 소스:** Node Exporter가 CPU/Memory/Disk/Network 등 호스트 메트릭을 `/metrics` 로 노출(9100)
- **수집 방식:** Prometheus가 `scrape_interval: 15s` 로 Pull
- **저장 방식:** Prometheus가 Mimir로 **Remote Write**하여 장기 저장(9009)
- **가치:** 단일 VM이라도 시계열 장기 보관이 가능하여 “용량 추세/성능 추세” 기반 운영 판단이 가능

2.2.3 트레이스 파이프라인: OTLP gRPC(4317) → Tempo → MinIO

- **수집 방식:** Tempo가 OTLP gRPC endpoint(4317)를 열고 애플리케이션/에이전트가 보낸 트레이스를 수신
- **저장 방식:** Tempo는 WAL(write-ahead log)을 로컬 볼륨에 기록하고, 최종적으로 S3 호환 스토리지(MinIO)로 trace block을 저장
- **가치:** MSA 전환 시 서비스 간 호출 체인을 TraceID로 추적하여 병목/오류 위치를 빠르게 식별 가능

2.3 포트 구성 및 보안 권한

요구사항서의 보안 권고에 따라 **Grafana(3000)만 외부 오픈**, 나머지 컴포넌트는 **Security Group**에서 외부 접근을 차단하고 Docker 내부 네트워크 `lgtm-net`에서만 통신하도록 구성한다.

컴포넌트	포트(컨테이너)	통신 방향	외부 오픈	비고
Grafana	3000	외부 브라우저 → Grafana	허용	대시보드/알림/데이터소스 통합
Loki	3100	Promtail → Loki	차단	LogQL 대상 로그 저장
Promtail	9080(관리 API)	내부(필요시)	차단	로그 수집 에이전트
Prometheus	9090	Prometheus → Node Exporter / Mimir	차단	Scrape + Remote Write 수행
Node Exporter	9100	Prometheus → Node Exporter	차단	호스트 메트릭 노출
Mimir	9009	Prometheus → Mimir / Grafana → Mimir	차단	장기 메트릭 저장(모놀리식)
Tempo	3200(HTTP), 4317(gRPC)	App/Agent → Tempo / Grafana → Tempo	차단(권장)	OTLP 수신은 내부에서만 사용
MinIO	9000(S3), 9001(Console)	Mimir/Tempo ↔ MinIO	차단	S3 호환 스토리지

3. 구축 과정(주차별 수행 내역 및 주요 설정)

3.1 주차별 수행 내역 요약

3.1.1 1주차 : 베이스 스택 기동 및 연동

- **핵심 목표:** 인프라 기초 확립 및 데이터 수집 파이프라인 연동
- **주요 수행 항목:** 인스턴스 생성, `docker-compose.yml` 및 컴포넌트 설정 파일 작성, 로그/메트릭/트레이스 파이프라인 완성, 대시보드 생성 및 알람(Alert)설정, Readme 작성
- **산출물 및 검증:** Grafana UI 접속 확인, Loki/Mimir/Tempo 데이터소스 연결 상태 검증, 모니터링 알람 설정 및 Git `output` 폴더 정리 완료

3.1.2 2~3주차: 고도화, 통합 안정화 및 문서화

- **핵심 목표:** 운영 안정성 확보, 보안 강화 및 마이그레이션 중심의 시스템 고도화
- **주요 수행 항목:** 컴포넌트 이미지 버전 고정, 컨테이너 Root권한 제거를 통한 보안 설정, Grafana Provisioning 자동화, 로그 샘플링 적용, Alloy 마이그레이션 테스트
- **산출물 및 검증:** 대시보드 및 알람 정상 동작 최종 검증, 핵심 트러블슈팅 정리, 결과보고서 작성

3.2 주요설정 설명

3.2.1 Promtail 주요 설정 및 파이프라인 요약(`promtail-config.yaml`)

- **기본 통신 및 수집 타겟 설정**
 - **수집 대상:** `/var/log/*log` 경로를 지정하여 마운트된 VM의 시스템 및 애플리케이션 로그를 타겟팅.
 - **로그 전송(client):** 수집된 로그 데이터를 내부 Loki 서버(`http://loki:3100`)로 지속해서 Push.
- **동적 라벨링 및 조건부 필터링**
 - **변수 및 레벨 추출(regex):** 로그 원문을 정규식으로 분석하여 보안 관련 주요 키워드(login, logout, auth 등) 포함 여부를 확인하고, 해당 로그의 레벨(INFO, ERROR 등)을 추출

- **동적 로그 분류(template & label):** 보안 키워드가 발견된 로그는 log_type을 `security` 로, 그 외의 로그는 `generic` 으로 분류한 뒤, 이를 Loki의 실제 검색 Label로 등록.
- **조건부 데이터 드롭(match):** 스토리지 효율성을 위해 일반로그(`generic`)이면서 로그레벨이 INFO인 경우 수집하지 않고 Drop.

3.2.2 Loki 데이터 저장 및 보관 정책 요약(`loki-config.yaml`)

- **기본 시스템 및 스토리지 구성**
 - **단일 노드 환경:** Replication Factor를 1로 설정하고 in-memory 방식을 사용하여, 클러스터링 없이 경량 구조로 구성
- **메모리 최적화 및 오래된 데이터 수신 제한(Ingestor & Limit)**
 - **Chunk 관리:** 메모리에 수집된 로그는 최대 1시간이 경과하거나 약 1MB 크기에 도달 할 때 스토리지에 기록되도록 설정.
 - **데이터 방어:** 수집 시점 기준으로 1년이상 비정상적인 과거 데이터가 인입 될 경우 수신 거부.
- **스트림별 차등 로그 보관 주기 (Retention Policy)**
 - **기본 보관 주기:** 로그의 기본 보관기간을 30일로 제한하여 스토리지 비용 절감.
 - **보안 로그 장기 보관:** 보안 키워드가 포함되어 `security` 라벨이 부여된 로그는 보안 감사 및 컴플라이언스 대응을 위해 1년 장기 보존.
- **데이터 파기(Compactor)**
 - **Compactor 활성화:** 설정된 Retention정책이 실제 반영되도록 Compactor는 백그라운드에서 만료된 로그를 주기적으로 삭제처리.

3.2.3 Prometheus 주요 설정 및 파이프라인 요약(`prometheus-config.yaml`)

- **기본 통신 및 수집 타겟 설정**
 - **데이터 수집:** 시스템 메트릭 수집주기 15초로 설정하여 고해상도 모니터링 설정.
 - **Mimir 원격 전송(Remote Write):** 수집된 모든 시계열 데이터를 Mimir(`http://mimir:9009`)서버로 Push
- **주요 수집 타겟**
 - **모니터링 시스템 자가 진단:** Prometheus 서버 자체를 수집 대상으로 지정하여, 수집기의 성능, 메모리, 사용량 등을 모니터링해 파이프라인의 건전성 확보.
 - **인프라/OS자원 모니터링:** 대상 서버에 배포된 Node Exporter를 통해 CPU Usage, Disk I/O등 OS레벨의 기초 메트릭 수집

3.2.4 Mimir 설정 및 보관 정책 요약(`mimir-config.yaml`)

- **기본 시스템 및 스토리지 구성**
 - **Monolithic Mode:** Ingestor, Compactor, Querier등의 마이크로 서비스를 통합실행.
 - **Minio(S3) 백엔드 :** 로컬 디스크의 용량 및 데이터 안정성을 위해 Object Storage 연동 및 시계열 블록 설정
- **메트릭 보관 주기 정책**
 - **Retention 정책:** 스토리지 운영 비용을 고려해 수집된 메트릭은 30일 동안 보관
 - **Compactor 설정:** 30일 경과 메트릭 블록은 Compactor가 백그라운드에서 삭제 처리.

3.2.5 Tempo 설정 및 보관 정책 요약(`Tempo-config.yaml`)

- **프로토콜 및 기본 시스템**
 - **단일 테넌트 환경**: 권한 분리 없이 단일 사용자 환경
 - **OTLP기반 트레이스 수집**: 글로벌 표준인 OpenTelemetry(OTLP) 프로토콜을 채택하고, gRPC 방식으로 애플리케이션의 분산 추적 데이터를 빠르고 안정적으로 수신.
- **유실방지 및 보관 정책**
 - **WAL(Write-Ahead Log) 적용**: 수집된 데이터가 Minio(S3)로 전송되기 전 로컬디스크에 우선 기록되도록 구성하여 장애시에도 유실 방지.
 - **Block_retention**: 트레이스 데이터는 단기간에 엄청난 용량이 쌓이기 때문에 트레이스 보관기간을 3일로 제한.
 - **Background Worker 설정**: 1시간 주기로 데이터 블록들을 병합 및 압축하여 스토리지 효율을 올리고 Retention정책이 지난 블록들을 폐기.

3.3 LGTM 옴저버빌리티 스택 고도화 세부 내역

3.3.1 환경 일관성 및 재현 안정성 확보: Image Version Pinning

배경: Docker 이미지 태그 중 `:latest` 는 편리하지만, 다음과 같은 운영 리스크가 있다.

- 재기동/재배포 시점에 **새 이미지가 자동으로 선택**될 수 있어, 동일한 구성 파일로도 동작이 달라진다.
- 트러블슈팅 시 “동일 문제 재현”이 어려워지고, 문서화된 환경과 실제 환경의 차이가 커진다.

조치: `docker-compose.yaml` 에서 이미지 태그를 명시적으로 고정했다.

- 실행 중 컨테이너의 바이너리/레지스트리 digest를 대조하여 실제 구동 버전을 식별하고, 문서/코드에 반영했다.

효과: 누구든 동일 리포지토리를 clone하고 compose up 하면 **동일 버전**으로 재현 가능

- 장애 발생 시 버전에 대한 변수를 제거하여 원인 분석이 단순해짐
- 결과보고서 및 README의 신뢰도가 상승

3.3.2 Grafana Provisioning 자동화: DataSource/Dashboard/Alert를 코드로 관리

배경: Grafana는 UI 기반으로 빠르게 대시보드/알림을 만들 수 있지만, 운영 환경에서는 다음 문제가 발생한다.

- 다른 환경(개발/스테이징/운영)으로 이관 시 수동 클릭이 반복되어 오류 가능성 증가
- 장애 복구(재배포) 시 초기 상태를 빠르게 복원하기 어려움

조치: Grafana의 provisioning 디렉터리를 컨테이너에 마운트하여 다음 자산을 코드로 관리하도록 구성했다.

- `provisioning/datasources`: Loki/Mimir/Tempo 데이터소스 자동 등록
- `provisioning/dashboards`: JSON 대시보드 자동 로드
- `provisioning/alerting`: Alert rule 자동 등록(파일 기반)

또한 환경 변수 기반 기능(`GF_PROVISIONING_ENV_VARS_ENABLE=true`)을 활용하여, 민감 정보(예: SMTP 계정)는 `.env` 에서 주입하고 코드에는 노출되지 않도록 분리했다.

효과: 재배포 시 **클릭 없이 동일한 대시보드/알림/데이터소스가 자동 복원**

- GitHub 커밋 히스토리로 변경 추적 가능
- 구성의 표준화로 신뢰성, 재현성 확보

3.3.3 Tempo 컨테이너 Root 권한 제거: Init 컨테이너 기법

배경 : Tempo는 WAL 및 설정 파일 접근을 위해 특정 경로(`/var/tempo/wal` , `/etc/tempo`)에 읽기/쓰기 권한이 필요하다. 단일 컨테이너에서 이를 단순히 해결하려고 `root` 로 실행하면 다음 문제가 발생한다.

- 컨테이너 내부 프로세스가 root 권한을 가지므로, 취약점 악용 시 피해 범위가 커진다.
- 호스트 마운트/볼륨이 있을 경우 권한 오남용 가능성이 증가한다.

조치 : `tempo-init` 컨테이너를 추가하여, 시작 시 필요한 디렉터리의 소유권을 Tempo 컨테이너 사용자 UID/GID `10001` 로 변경한 후 종료되게 했다.

- 이후 `tempo` 본 컨테이너는 `root` 로 실행하지 않도록 권한을 제거했다.

효과 : 최소 권한 원칙(Least Privilege) 준수로 보안 리스크 감소

- 권한 문제로 컨테이너가 즉시 종료되는 장애의 재발 방지
- "Init 컨테이너로 선작업 → 본 서비스는 최소 권한"이라는 운영 패턴을 학습

3.3.4 로그 레벨 기반 샘플링: INFO 로그 Drop

배경 : 운영/실습 환경에서 애플리케이션 및 시스템 로그는 `INFO` 레벨의 정상 동작 로그가 대다수를 차지한다. 이를 모두 수집/적재하면 다음과 같은 문제가 발생한다.

- Loki 저장소/디스크 사용량이 빠르게 증가하여 **retention 기간 내에도 용량 압박**이 생길 수 있다.
- Grafana에서 LogQL 조회 시 **불필요한 노이즈**가 증가해, 장애/이상 징후(예: ERROR/WARN) 탐지가 느려진다.
- 단일 VM 환경에서는 IO/CPU 자원이 제한적이므로, 로그 수집/전송 비용이 커질수록 전체 스택 안정성에 영향을 줄 수 있다.

따라서 필요한 로그(ERROR/WARN)는 남기되, 상대적으로 가치가 낮은 INFO 로그는 샘플링/필터링 하여 효율을 높이는 전략이 필요하다.

조치 : Promtail의 `pipeline_stages` 를 사용해 로그 본문에서 JSON 필드의 `"level":"INFO"` 값을 정규식으로 추출한 뒤, `INFO` 로그만 drop 하도록 구성하였다.

효과

- INFO 로그 적재량이 감소하여 **저장 비용 및 디스크 사용량을 절감**하고 retention 정책 운영이 용이해진다.
- LogQL 조회 시 노이즈가 줄어 **WARN/ERROR 중심의 트러블슈팅 속도**가 향상된다.

3.3.5 데이터 보존 정책(Retention) 수정

배경.

- **스토리지 자원의 한계:** 과거의 로그, 메트릭, 트레이스 데이터가 지속적으로 누적될 경우, 디스크 풀(Disk Full)로 인해 전체 모니터링 스택이 마비되는 장애가 발생할 수 있다.
- **데이터 특성별 가치 차이:** 로그, 메트릭, 트레이스별로 필요한 최소·최대 보존 기간이 다르다. 특히 용량이 큰 트레이스(Trace)나 단순 모니터링 메트릭은 상대적으로 수명이 짧은 반면, 특정 보안 로그 등은 법적/관리적 이유로 장기 보관이 요구된다.

- **비용 및 성능 최적화:** 무조건적인 장기 보관 대신 데이터의 성격과 관리 목적에 맞는 명확한 주기(Retention)를 부여하여, 디스크 비용을 절감하고 데이터 쿼리(Query) 조회 성능을 향상시키는 기준이 필요하다.

조치

데이터 종류 (Tool)	기본 보관 기간	특정 조건 보관 기간	설정 위치 및 주요 옵션
로그 (Loki)	30일 (720h)	보안 로그: 1년 (8760h)	<code>limits_config.retention_stream</code> selector: <code>'{log_type="security"}'</code>
메트릭 (Mimir)	30일 (30d)	-	<code>limits.compactor_blocks_retention_period</code>
트레이스 (Tempo)	3일 (72h)	-	<code>backend_worker.compaction.block_retention</code>

효과

- **디스크 관리의 예측 가능성 확보:** 데이터가 설정된 주기에 따라 자동으로 만료 및 정리되므로, 수동 삭제 작업 없이도 VM 내부 스토리지 용량을 안정적으로 유지할 수 있다.
- **중요 데이터 유실 방지 및 보안 규정 준수:** 일반 로그는 짧게 유지하여 용량을 아끼는 동시에, 보안 로그(`log_type="security"`)와 같은 핵심 데이터는 Long-term Retention(1년)을 적용함으로써 보안 감사 및 규정 준수 요건을 충족했다.
- **조회 속도 및 시스템 리소스 최적화:** 불필요하게 비대해진 과거 데이터 블록(Block)이 정리되면서 Grafana 대시보드 로딩 및 대용량 쿼리 분석 시 인덱스 검색 범위가 줄어들어 트러블슈팅 속도가 빨라진다.

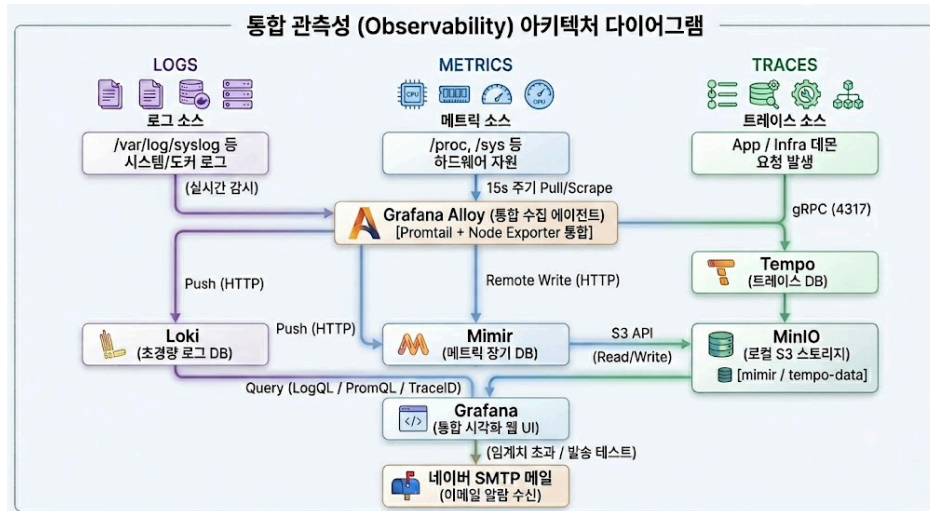
3.3.6 Promtail, Prometheus – Grafana Alloy 마이그레이션 테스트

배경

- Grafana는 Promtail 사용자에게 **Grafana Alloy**로의 마이그레이션을 권장하고 있다.
- 또한 향후 쿠버네티스 기반 환경으로 전환할 경우, 메트릭, 트레이스, 로그 수집을 **단일 에이전트로 통합**하는 방향이 운영 측면에서 더 유리하다고 판단하여 사전 검증을 수행했다.

조치

- **운영 안정성 보장:** 기존 `main` 브랜치는 그대로 유지하고, 별도의 `alloy` 브랜치를 생성하여 변경을 격리했다.
- **검증 환경 분리:** 신규 VM 인스턴스를 추가로 생성하여(접속 IP: `1.201.177.162`) 테스트 환경을 완전히 분리한 뒤, 해당 인스턴스에서 마이그레이션 작업을 진행했다.
- **마이그레이션 수행 및 성공:** `Promtail` + `Prometheus` 기반 수집 구성을 제거하고, **Grafana Alloy 단일 프로세스**로 메트릭/로그 수집 파이프라인을 구성하여 마이그레이션을 완료했다.
- **검증 채널**
 - Alloy UI: <http://1.201.177.162:12345/>
 - Grafana UI: <http://1.201.177.162:3000>



효과 및 의의

- **에이전트 운영 단순화:** 기존에는 로그 (Promtail)와 메트릭 (Prometheus)을 각각 운영해야 했으나, Alloy로 통합하면 관리 대상이 1개로 축소되어 운영 부담이 감소한다.
- **수집 파이프라인 일원화:** 메트릭 스크래핑과 로그 파이프라인을 한 곳에서 정의·관리할 수 있어, 설정 불일치 리스크를 낮출 수 있다.
- **K8s 전환 시 ConfigMap 관리 단순화:** 클러스터 환경에서는 노드 단위로 수집기를 배포하게 되는데, Alloy 기반으로 통합하면 권한(RBAC)/ConfigMap/배포 정책을 단일 수집기로 표준화할 수 있어 운영 효율성이 향상된다.
- **디버깅 가시성 향상(Alloy UI 기반):** Alloy 내장 UI를 통해 컴포넌트 간 데이터 흐름을 확인할 수 있어, 장애 발생 시 “백엔드 문제”인지 “수집단 문제”인지의 구분이 빨라져 트러블슈팅 시간을 단축할 수 있다.

4. 검증 결과

4.1 서비스 정상 구동 상태(docker compose ps)

서비스가 정상 구동(Up) 상태 확인. Compose 상 서비스 목록은 아래와 같다.

```
ubuntu@vm-project1-1g1m:~/LGTM_Observability_stack$ docker compose ps
```

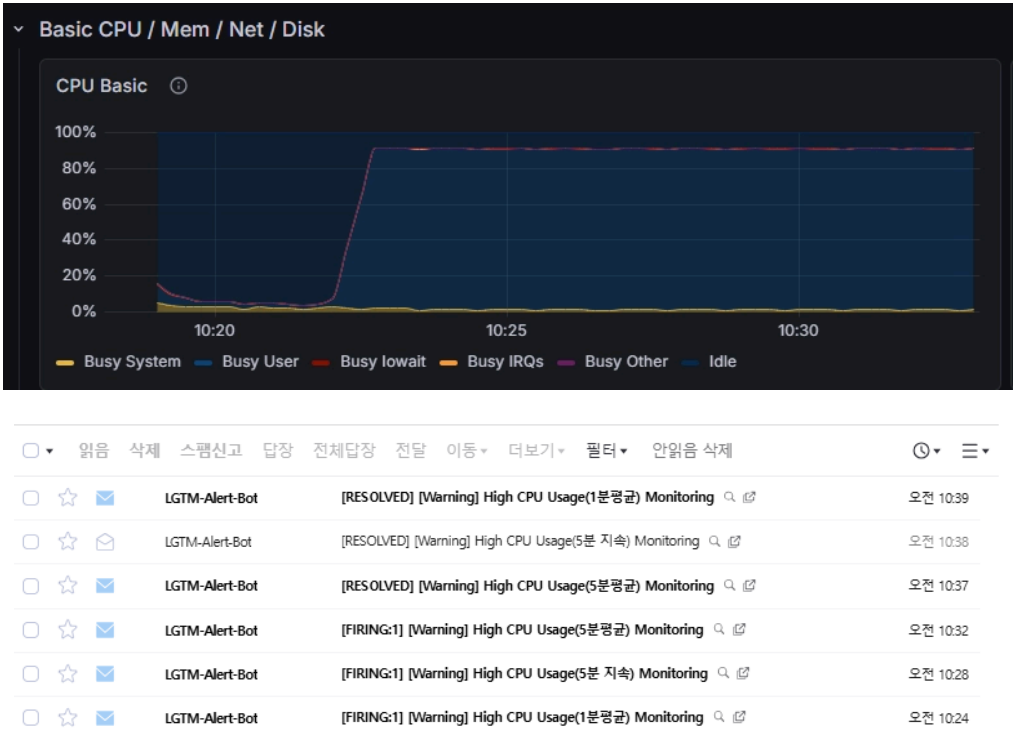
NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS
grafana	grafana/grafana:13.0	"/run.sh"	grafana	4 days ago	Up 4 days
loki	grafana/loki:3.7.2	"/usr/bin/loki -conf..."	loki	4 days ago	Up 4 days
mimir	grafana/mimir:3.1.0	"/bin/mimir -config..."	mimir	4 days ago	Up 4 days
minio	minio/minio	"/usr/bin/docker-ent..."	minio	4 days ago	Up 4 days (healthy)
node-exporter	prom/node-exporter:v1.11.1	"/bin/node_exporter ..."	node-exporter	4 days ago	Up 4 days
prometheus	prom/prometheus:v3.12.0	"/bin/prometheus --c..."	prometheus	4 days ago	Up 4 days
promtail	grafana/promtail:3.6.8	"/usr/bin/promtail -..."	promtail	4 days ago	Up 4 days
tempo	grafana/tempo:3.0.0	"/tempo -config.file..."	tempo	4 days ago	Up 4 days

4.2 장애 알람 시나리오

4.2.1 CPU 부하 테스트

- **알람 조건:**
 - CPU Usage > 80% 5분 지속 시
 - CPU Usage 5분 평균값 > 80% 5분 지속 시
 - CPU Usage 1분 평균값 > 80% 1분 지속 시


```
ubuntu@vm-project1-1g7m:~/LGTm_Observerbility_stack$ stress-ng --cpu 4 --cpu-load 90 --timeout 720s
stress-ng: info:  [2647189] setting to a 720 second (12 mins, 0.00 secs) run per stressor
stress-ng: info:  [2647189] dispatching hogs: 4 cpu
```

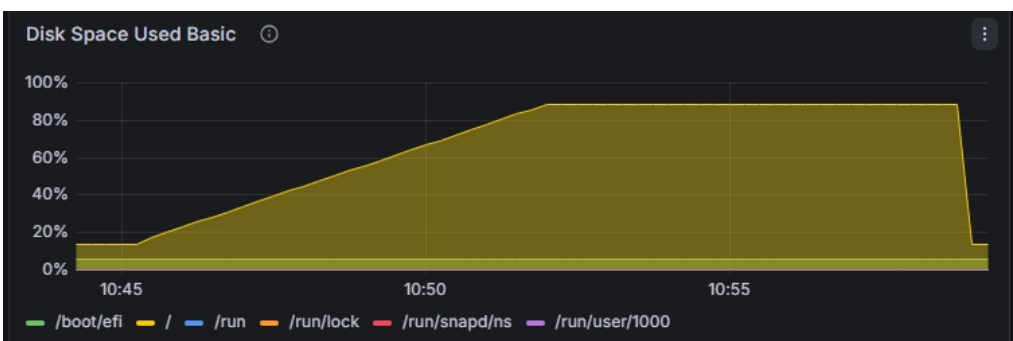


4.2.1 DISK 부하 테스트

- **알람 조건**

- DISK Usage > 85% 수치 상회 즉시
- DISK Usage > 85% 5분 지속 시

```
ubuntu@vm-project1-igtm:~/LGTM_Observerbility_stack$ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/vda1       49G   6.6G   42G   14% /
ubuntu@vm-project1-igtm:~/LGTM_Observerbility_stack$ sudo dd if=/dev/zero of=/tmp/disk_test_
file bs=1G count=36 status=progress
38654705664 bytes (39 GB, 36 GiB) copied, 400 s, 96.6 MB/s
36+0 records in
36+0 records out
38654705664 bytes (39 GB, 36 GiB) copied, 400.294 s, 96.6 MB/s
ubuntu@vm-project1-igtm:~/LGTM_Observerbility_stack$ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/vda1       49G   43G   5.7G   89% /
```



			LGTM-Alert-Bot	[RESOLVED] [Critical] High Disk Usage(즉시 알림) Monitoring (/dev/vda1 ext4 node-exporter9100 node-exporter /) 🔍 📄	오전 11:02
			LGTM-Alert-Bot	[RESOLVED] [Warning] High Disk Usage Monitoring (/dev/vda1 ext4 node-exporter9100 node-exporter /) 🔍 📄	오전 11:02
			LGTM-Alert-Bot	[FIRING:1] [Warning] High Disk Usage Monitoring (/dev/vda1 ext4 node-exporter9100 node-exporter /) 🔍 📄	오전 10:57
			LGTM-Alert-Bot	[FIRING:1] [Critical] High Disk Usage(즉시 알림) Monitoring (/dev/vda1 ext4 node-exporter9100 node-exporter /) 🔍 📄	오전 10:52

5. 트러블슈팅

5.1 MinIO 준비 전 종속 컨테이너 초기화 실패

1) 현상

- `create-bucket`, `mimir`, `tempo` 컨테이너가 오작동.
- 스택 전체 기동이 불안정해지고 “재시작 반복” 또는 “버킷 생성 실패”가 반복됨.

2) 원인(Root Cause)

- Docker Compose는 기본적으로 컨테이너 시작 순서만 제어할 뿐, 서비스 준비 완료를 보장하지 않는다.
- MinIO는 프로세스 시작 이후에도 내부 초기화 시간이 필요하며, 그 사이 종속 서비스가 S3 endpoint를 호출하면 실패한다.

3) 해결(Resolution)

- MinIO에 `/minio/health/live` 기반 healthcheck를 추가하여 “Ready”를 판별
- `depends_on`에 `condition: service_healthy`를 적용하여 MinIO가 healthy 상태가 된 후에만 종속 컨테이너가 실행되도록 통제

4) 결과(Outcome)

- MinIO 준비 완료 이후에만 종속 서비스가 시작되어 초기화 실패가 제거됨
- 버킷 생성 및 S3 연동 안정성이 확보되어 장기 운용 시 재현 가능성이 향상됨

5.2 Grafana 대시보드 연결 실패

1) 현상(Symptom)

- 데이터 소스(Data Source) 설정 화면에서 'Save & Test' 실행 시 정상(Success)으로 연동됨을 확인.
- 하지만 실제 메트릭 데이터 대시보드상에는 데이터가 시각화되지 않거나, 연결이 끊어진 것처럼 빈 화면(No Data)이 노출됨.

2) 원인(Root Cause)

- 과거에 사용하던 기존 Job 이름들이 데이터 소스나 수집기 측에 여전히 다수 남아 있는 상태 확인.
- Save & Test는 데이터 소스 엔진 자체의 통신상태만 검증하기 때문에 정상으로 통과했으나, 실제 대시보드 패널들은 변경된 최신 Job 이름을 바라보지 못하고 과거의 Job이름으로 메트릭을 조회함.

3) 해결(Resolution)

- Job 이름 정제 및 매핑 수정
 - 대시보드 환경설정 및 쿼리문을 점검하여 과거 이름으로 박혀있던 레거시 Job명칭을 업데이트함
- 불필요한 레거시 메트릭 정리

4) 결과(Outcome)

- 대시보드 패널과 실제 수직되는 메트릭 Job이름이 일치되면서, 대시보드 상에 데이터가 정상적으로 실시간 시각화 됨을 확인

5.3 Grafana 대시보드 프로비저닝 실패

1) 현상(Symptom)

- Grafana 파일 기반 대시보드 프로비저닝 적용 시, JSON 파일이 “유효하지 않은 파일”로 판단되어 자동 로드가 차단됨
- UI에서는 대시보드가 보이지 않음

2) 원인(Root Cause)

- Grafana UI에서 Export한 대시보드가 웹사이트 Import용인 `V2 Resource` 로 추출됨
- 파일 프로비저닝 엔진은 `V2 Resource` 스키마 구조를 기대하지 않아 JSON을 정상 파싱하지 못한다.

3) 해결(Resolution)

- Grafana 대시보드 Export 시 **Advanced options**로 진입하여
 - Model을 `Classic` 으로 전환
 - `Classic` 모드에서 **Raw JSON** 형태로 Export
- Export한 Classic Raw JSON을 provisioning 대상 경로에 배치 후 Grafana 재기동

4) 결과(Outcome)

- 수동 클릭/우회 API 없이, 컨테이너 기동 시점에 대시보드가 자동으로 로드되는 것을 확인
- 환경 재현성이 향상됨

5.4 Tempo v3.0 기동 실패

1) 현상(Symptom)

- Tempo v3.0 에서 데이터 보존 정책(Retention) 추가시 Tempo 컨테이너가 정상적으로 뜨지 못하고 무한 재기동 (`Restarting`)되는 현상 발생
- 컨테이너 로그 확인 시 `compactor_config` 필드를 파싱할 수 없다는 문법 오류 발생
- 대다수의 기존 블로그, 기술 레퍼런스 및 생성형 AI모델들은 v2.x이하의 `compactor_config` 구형 문법을 제시해 문제 해결에 도움이 되지 않음

2) 원인(Root Cause)

- **.Grafana Tempo 3.0의 아키텍처 변경:**
 - Tempo가 v3.0 버전으로 올라오면서 기존의 데이터 압축 및 보존(Retention)을 담당하던 `compactor` 모듈이 제거됨
 - 3.0 버전부터는 더욱 효율적인 분산 처리를 위해 Job 기반의 `backend scheduler` 와 `backend worker` 아키텍처로 대체됨

3) 해결(Resolution)

- Grafana Tempo v3.0 공식 가이드 및 릴리스 노트를 참조하여 기존의 `compactor` 설정을 완전히 제거
- `backend_worker` 구조를 도입하고, 하위에 `compaction` 및 `block_retention` 설정을 정의하여 데이터 보존(Retention) 정책을 v3.0 규격에 맞게 수정

4) 결과(Outcome)

- 문법 오류가 사라지며 **tempo** 컨테이너가 **Up** 상태로 정상 가동됨을 확인

6. 회고 및 다음 단계

6.1 배운 점

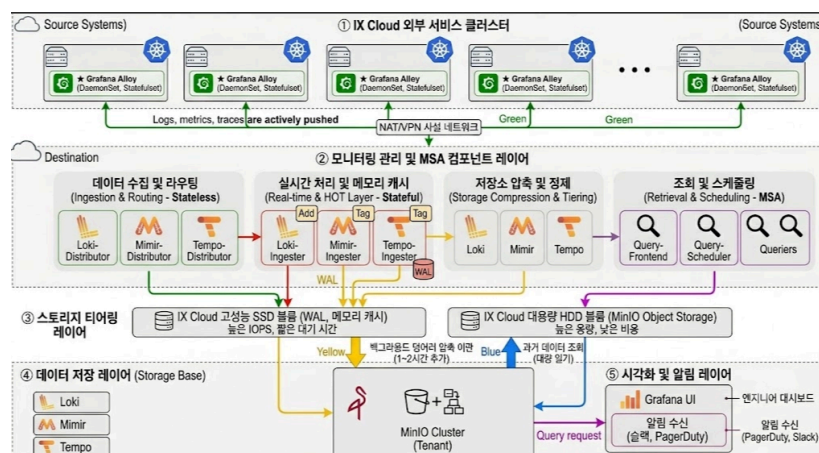
- Logs/Metrics/Traces는 각각 단독으로도 가치가 있지만, MSA 환경에서의 문제 해결 속도는 **3개 신호를 상호 연계**할 때 비약적으로 개선된다.
 - 예: CPU 급증(메트릭) 시점의 오류 로그(로그), 특정 요청의 병목 구간(트레이스)까지 한 번에 연결해 추적 가능
- Docker Compose 기반 단일 VM 실습이라도, **의존성(healthcheck/depends_on)**, **권한(least privilege)**, **구성 자동화(provisioning)**를 적용하면 품질을 크게 끌어올릴 수 있음을 느꼈다.
- 이 프로젝트에서는 **:latest 태그**를 붙여서 작업을 하고, 후에 **버전정보**를 추출하여 고정하는 방식을 사용하였으나 다음부터는 각 **기술스택의 버전정보**를 미리 확인하고 고정한 후 구축해야 품질을 더 높일 수 있음을 느꼈다.

6.2 한계점

- 단일 VM + Docker Compose 환경이므로, 실제 MSA 환경처럼 컴포넌트를 분산 배치하거나 HA/Replica 기반 확장을 실행하기 어렵다.
- 애플리케이션 연동이 제한적이어서, 트레이스는 인프라 측면에서의 기반 구축 위주로 검증되었다.

6.3 다음 단계

- **Kubernetes(K8s) 전환**: Docker Compose → K8s로 이관하고 Helm 기반(loki-distributed, mimir-distributed, tempo-distributed)으로 분산화를 구현
- **컴포넌트 분산**: Distributor, Ingester, Compactor등의 컴포넌트들을 분산하고 세팅하여 MSA화
- **오토스케일링/스토리지 확장**: HPA/VPA, object storage 확장, retention/compaction 정책을 실제 운영 수준으로 고도화
- **실제 애플리케이션 연동**: Spring Boot/Node.js 서비스에 OpenTelemetry SDK를 적용하여 “요청 단위 장애 분석” 트레이싱을 경험 및 강화



7. 종합 및 결론

본 프로젝트는 단일 Cloud VM 환경 내에서 LGTM(Loki, Grafana, Tempo, Mimir) 기반의 통합 관찰성 파이프라인을 성공적으로 구축하고 고도화하였습니다.

- **관찰성 통합 체계 마련:** 분산되어 있던 로그, 메트릭, 트레이스를 Grafana 단일 뷰포트로 통합함으로써 시스템 이상 징후 감지부터 원인 분석까지의 과정을 단일 워크플로우로 연계할 수 있는 기반을 다졌습니다.
- **엔지니어링 완성도 확보:** 단순 기능 구현에 그치지 않고, 이미지 버전 고정(Pinning), Grafana 프로비저닝 자동화, 인프라 보안 권한 최적화, Promtail의 로그 샘플링 정책 등을 적용하여 실제 운영 환경에 준할 수 있도록 노력하였습니다.
- **미래 아키텍처 다변화 검증:** 2주차에 진행된 Grafana Alloy로의 마이그레이션 테스트를 통해 수집 에이전트의 일원화 가능성을 확인하였으며, 이는 향후 쿠버네티스(K8s) 클러스터 환경으로의 확장 및 모니터링 컴포넌트 관리를 효율화하는 최적의 디딤돌이 될 것입니다.

본 프로젝트를 통해 확보한 오픈소스 기반의 모니터링 설계 역량과 트러블슈팅 경험은 시스템 엔지니어로서 중단 없는 서비스 운영을 위한 견고한 기술적 자산이 될 것으로 기대됩니다.

[이상 보고서 끝]